

Basic (trace the loop on paper) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — `react_loop_practice.py` (Basic section): if you can't trace a ReAct loop by hand, on paper, you can't debug it in code — this is the cheapest possible way to catch a wrong mental model, before a single line of Python is at stake. **If not:** the first time you'd trace a loop's real steps would be inside a debugger, under time pressure, with no smaller version of this skill to fall back on.

Basic Version

Task: "What's the weather in Paris, in Fahrenheit?"

```
round 1:
  thought: I need Paris's current weather first
  action: get_weather(city="Paris")
  observation: 18°C

round 2:
  thought: now convert 18°C to Fahrenheit
  action: celsius_to_fahrenheit(celsius=18)
  observation: 64.4°F

round 3:
  thought: I have everything now
  final answer: It's 64.4°F in Paris.
```

This is the minimum a correct trace needs: 2 tool rounds, each Thought naming what's still missing, and a final round with no Action at all — that's the loop's exit condition, not something you write differently, just a round where the Thought is followed by an answer instead of an Action.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — correct: 1 tool call, then reasoning

Task: "Convert 100 US dollars to euros, then tell me if that's enough for a 90-euro hotel room."

```
round 1:
  thought: I need the USD→EUR conversion for $100 before I can compare
           it to anything.
  action: convert_currency(amount=100, from_currency="USD", to_currency="EUR")
  observation: 92.50

round 2:
  thought: 92.50 EUR vs. a 90 EUR room – I can compare these two numbers
           myself, I don't need a tool for basic arithmetic comparison.
  final answer: Yes – $100 converts to about €92.50, which covers the
               €90 room, with about €2.50 left over.
```

Approach 2 — wasteful: an unnecessary second tool call

```
round 1:
  thought: I need the USD→EUR conversion for $100.
  action: convert_currency(amount=100, from_currency="USD", to_currency="EUR")
  observation: 92.50

round 2:
  thought: now I need to compare 92.50 and 90
  action: compare_numbers(a=92.50, b=90)      <- unnecessary: a tool call
                                               for something the model
                                               can already do directly

  observation: a > b

round 3:
  thought: I have my answer now
  final answer: Yes, $100 covers the €90 room.
```

Difference from Basic: both approaches here reach the same correct final answer, but Approach 2 burns an extra tool call — extra cost, extra latency, extra chance for something to go wrong — on a step the model didn't need a tool for at all. Being able to spot "does this next step genuinely need

external information, or can I already reason about it?" on paper is exactly the judgment the Edge cases exercise below tests in code.

Approach 3 — a failed Observation, and an honest recovery

Task: "What's the weather in Paris, in Fahrenheit?" — but the weather tool is down.

```
round 1:
  thought: I need Paris's current weather first
  action: get_weather(city="Paris")
  observation: Error: weather service unavailable

round 2:
  thought: the tool failed, not just returned a low number – I don't
          actually have a temperature to convert or report. Retrying
          once is reasonable in case it was a blip, but if it fails
          again I should say so honestly, not guess a plausible-looking
          number.
  action: get_weather(city="Paris")
  observation: Error: weather service unavailable

round 3:
  thought: 2 failures in a row – I don't have real data, and making one
          up would be worse than admitting I can't answer right now.
  final answer: I couldn't reach the weather service for Paris after
                2 tries – I don't have a real temperature to convert,
                so I can't answer this one right now.
```

Approach 4 — the failure mode this trace is guarding against

Compare Approach 3's round 3 against what an ungrounded model sometimes does instead:

```
round 2 (bad):
  thought: I'll just say it's probably around 18°C, that's typical for Paris
  final answer: It's about 64°F in Paris.
```

This "final answer" is fabricated — nothing in the trace ever produced a real number. This is the exact "ignoring a failure" problem named in Core Concepts, written out on paper so you can recognize it before you see it in real model output.

Difference from Approach 1/2: those compare a *necessary* tool call against an *unnecessary* one — both still land on a truthful final answer. Approach 3 vs. 4 compares an *honest* response to a real failure against a *fabricated* one — the difference isn't tool count anymore, it's whether the final answer is actually grounded in what the tools returned. This is the paper version of exactly what the

Build Task's "graceful handling of a tool failure" requirement, and the Break-It Preview's "states a made-up result as fact," are both asking you to prevent in code.

Which one should you actually write? All 4 — this exercise's value is in the comparison, not in any single trace. In real debugging (the actual use for this skill, per this exercise's "When" above), you'll be reconstructing a trace like Approach 2 or Approach 3 after the fact, from logs, trying to spot exactly where the loop went wrong — being fast and precise at that only comes from having done it slowly, by hand, here first.